



Memoire de fin de module pour la validation du module Full-Stack

Filière : Développement Informatique - S6 (DEV-INFO S6)

Sujet :

E-STORE

*Conception et réalisation d'une plateforme
e-commerce full-stack à persistance hybride
(Spring Boot - Angular - JPA - MongoDB)*

Encadre par :

Pr. ZAHOUR Omar

Réalise par :

Akram BELMOUSSA
Nouhaila BEN SOUMANE

Soutenu le 10 Mai 2026, devant le jury :

Pr. ZAHOUR Omar

Encadrant

Pr. BEN Lahmar El Habib

Examineur

Pr. EL KANDALI Khalid

Examineur

Année universitaire : 2025 - 2026

Dedicace

*C'est avec profonde gratitude et sinceres mots, que nous dedions ce modeste travail de fin de module a nos chers **parents**, qui ont sacrifie leur vie pour notre reussite et nous ont eclaire le chemin par leurs conseils judicieux.*

Nous esperons qu'un jour, nous serons leur source de fierte et de bonheur, et que Dieu leur prete une longue et heureuse vie.

*Nous dedions egalement ce travail a nos **freres et soeurs**, a nos **familles**, a nos **amis**, et a tous nos **chers professeurs** qui nous ont accompagnes tout au long de notre parcours.*

Akram & Nouhaila

Remerciements

Au terme de ce projet, il nous est tres agreable d'exprimer nos sincereres remerciements a toutes les personnes qui ont contribue, de pres ou de loin, a la realisation de ce travail.

*Nous tenons en premier lieu a remercier **Allah** le tout puissant de nous avoir donne la sante, la volonte et le courage d'entamer et de terminer ce projet.*

*Nous adressons nos plus vifs remerciements a notre encadrant pedagogique, **Pr. ZAHOUR Omar**, pour la qualite de son enseignement, sa disponibilite permanente, ses conseils avises et la rigueur scientifique qu'il a su nous transmettre tout au long de ce module. Ses orientations ont ete determinantes dans la definition du perimetre du projet et dans les choix techniques que nous avons operes.*

*Nous remercions egalement l'ensemble du **corps professoral** du Departement de Mathematiques et Informatique de la Faculte des Sciences Ben M'Sik - Universite Hassan II de Casablanca, dont les enseignements anterieurs en programmation orientee objet, en bases de donnees, en genie logiciel et en architectures web ont constitue les fondations sur lesquelles ce travail a pu s'elever.*

*Nos remerciements vont enfin a nos **familles** et a nos **camarades de promotion DEV-INFO S6**, pour leur soutien constant, leurs relectures attentives et l'emulation intellectuelle qui nous a tires vers le haut.*

*Nous adressons enfin nos remerciements anticipes aux **membres du jury** qui nous font l'honneur d'evaluer ce travail.*

Resume

*Ce memoire presente la conception et la realisation de **E-Store**, une application e-commerce complete developpee dans le cadre du module Full-Stack de la filiere DEV-INFO Semestre 6 a la Faculte des Sciences Ben M'Sik. Le projet adopte une architecture rigoureuse en trois couches techniques (presentation, logique metier, acces aux donnees) et un decoupage en cinq domaines fonctionnels suivant les principes du Domain-Driven Design.*

*L'application implemente l'ensemble du parcours utilisateur d'un site marchand : inscription, authentication, navigation dans un catalogue, gestion d'un panier, validation transactionnelle de commandes et depot d'avis produits. La particularite technique majeure reside dans la mise en oeuvre d'une **persistance hybride** associant un systeme relationnel (MySQL ou H2) pour les donnees transactionnelles a un systeme documentaire (MongoDB) pour les avis utilisateurs.*

Le backend repose sur Spring Boot 3.3 et Spring Security 6 avec une authentication stateless par jeton JWT. Le frontend exploite les nouveautes d'Angular 17 (composants standalone, signals, lazy loading) et le framework Bootstrap 5. Une suite de neuf tests unitaires JUnit 5 / Mockito couvre les services metier critiques.

Mots-cles : *Spring Boot, Angular, JWT, MongoDB, Architecture en couches, Domain-Driven Design, Persistance hybride, ACID, REST, BCrypt.*

Abstract

*This dissertation presents the design and implementation of **E-Store**, a complete e-commerce application developed as part of the Full-Stack module of the DEV-INFO S6 program at the Faculty of Sciences Ben M'Sik. The project follows a rigorous three-layer architecture (presentation, business logic, data access) combined with a five-domain decomposition inspired by Domain-Driven Design.*

*The application covers the entire customer journey of a merchant website: registration, authentication, catalog navigation, cart management, transactional order validation and product review submission. The main technical originality lies in the use of a **hybrid persistence** model combining a relational database (MySQL or H2 in-memory) for transactional data with a document database (MongoDB) for user reviews.*

The backend is built on Spring Boot 3.3 and Spring Security 6 with stateless authentication via JWT tokens. The frontend leverages the latest features of Angular 17 (standalone components, signals, lazy loading) combined with Bootstrap 5. A suite of nine unit tests (JUnit 5 / Mockito) ensures the reliability of the core business services.

Keywords: Spring Boot, Angular, JWT, MongoDB, Layered architecture, Domain-Driven Design, Hybrid persistence, ACID, REST, BCrypt.

Sommaire

Introduction generale	1
Chapitre 1 : Contexte et cahier des charges	3
1.1. Contexte academique	3
1.2. Etude du marche de l'e-commerce au Maroc	4
1.3. Cahier des charges fonctionnel	5
1.4. Cahier des charges non fonctionnel	6
1.5. Methodologie de travail	7
Chapitre 2 : Architecture et conception	8
2.1. Architecture en trois couches	8
2.2. Decoupage en domaines fonctionnels	9
2.3. Modele conceptuel de donnees	10
2.4. Modele documentaire MongoDB	11
2.5. Architecture de l'API REST	12
Chapitre 3 : Choix techniques	13
3.1. Backend : Java et Spring Boot	13
3.2. Persistance hybride MySQL + MongoDB	14
3.3. Frontend : Angular 17 standalone	15
Chapitre 4 : Realisation du backend	16
4.1. Structure du projet	16
4.2. Domaine customer (User, Profile, JWT)	17
4.3. Domaine catalog	18
4.4. Domaine inventory	19
4.5. Domaines shopping et billing	19
4.6. Domaine review (MongoDB)	21
4.7. Gestion globale des erreurs	22
Chapitre 5 : Securite	23
5.1. Hachage BCrypt des mots de passe	23
5.2. Authentification stateless par JWT	23
5.3. Autorisation par role	25
5.4. Configuration CORS	25
Chapitre 6 : Realisation du frontend	26
6.1. Structure de l'application Angular	26
6.2. Bootstrap de l'application	27
6.3. Routing et lazy loading	27
6.4. AuthInterceptor et ErrorInterceptor	28
6.5. AuthGuard et composants	29

Chapitre 7 : Tests et deploiement	30
7.1. Strategie de tests	30
7.2. Resultats d'execution	31
7.3. Conteneurisation Docker	32
Chapitre 8 : Difficultes et resultats	33
8.1. Difficultes rencontrees	33
8.2. Bilan fonctionnel et metriques	35
Conclusion generale et perspectives	36
Bibliographie	38
Annexes	39

Table des figures

Figure 1 : Architecture en trois couches de l'application E-Store	8
Figure 2 : Les six domaines fonctionnels	9
Figure 3 : Modele conceptuel de donnees relationnel	10
Figure 4 : Structure d'un document Review (MongoDB)	11
Figure 5 : Cycle de vie d'un JWT	24
Figure 6 : Resultats d'execution des tests JUnit	31
Figure 7 : Organisation Docker Compose	32

Liste des tableaux

Tableau 1 : Cas d'usage fonctionnels couverts par E-Store	5
Tableau 2 : Matrice architecturale (couches x domaines)	9
Tableau 3 : Endpoints REST principaux	12
Tableau 4 : Comparaison MySQL vs MongoDB	14
Tableau 5 : Suite de tests unitaires	30
Tableau 6 : Metriques quantitatives du projet	35

Liste des abreviations

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BCrypt	Blowfish Crypt (algorithme de hachage)
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
DDD	Domain-Driven Design
DEV-INFO	Developpement Informatique (filiere)
DTO	Data Transfer Object
FSBM	Faculte des Sciences Ben M'Sik
H2	Base de donnees relationnelle in-memory en Java
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCD	Modele Conceptuel de Donnees
NoSQL	Not Only SQL
REST	Representational State Transfer
S6	Semestre 6
SGBD	Systeme de Gestion de Bases de Donnees
SPA	Single Page Application
UI/UX	User Interface / User eXperience

CHAPITRE INTRO

Introduction generale

L'avenement du commerce electronique a profondement remodele les pratiques commerciales mondiales. Selon les estimations de la Confederation Marocaine du Commerce Electronique, le volume des transactions en ligne au Maroc a franchi le cap des **20 milliards de dirhams** en 2024, avec une croissance annuelle moyenne superieure a 25%. Cette dynamique s'accompagne d'une exigence accrue en termes d'ergonomie, de performance et de securite des plateformes marchandes.

Le module **Full-Stack** du **Semestre 6 de la filiere DEV-INFO**, dispense sous la direction du Pr. Omar Zahour, vise a doter les etudiants des competences necessaires a la conception et a la realisation de telles applications de bout en bout - du modele de donnees jusqu'a l'interface utilisateur, en passant par la logique metier, la securite et le deploiement.

C'est dans ce contexte que s'inscrit le present projet, baptise **E-Store**. Il s'agit d'une mise en pratique exhaustive des enseignements theoriques par la construction d'une application e-commerce complete, integrant les meilleurs standards de l'industrie en 2026 : Spring Boot 3.3, Angular 17, persistance hybride relationnel/documentaire, authentication JWT, tests unitaires automatises et conteneurisation Docker.

1. Objectifs du projet

Ce projet poursuit cinq objectifs principaux :

- **Maitriser une architecture full-stack moderne** en appliquant un decoupage en couches et en domaines fonctionnels rigoureux.
- **Illustrer la persistance hybride** en faisant cohabiter, au sein d'une meme application, un SGBD relationnel (MySQL) et un SGBD documentaire (MongoDB).
- **Appliquer les bonnes pratiques** de genie logiciel : DTOs, transactions, validation, gestion centralisee des erreurs, tests unitaires.
- **Securiser l'application** par un mecanisme stateless reposant sur Spring Security 6, BCrypt et JWT.
- **Produire une interface utilisateur reactive** avec Angular 17 en mode standalone, Bootstrap 5 et signals.

2. Plan du memoire

Le present memoire s'articule autour de **huit chapitres** :

- Le **chapitre 1** pose le contexte du projet et formalise le cahier des charges fonctionnel et non fonctionnel.

- Le **chapitre 2** decrit l'architecture generale, le decoupage en couches et en domaines, ainsi que la modelisation des donnees.
- Le **chapitre 3** justifie les choix techniques operes pour chaque brique de la stack.
- Les **chapitres 4 et 5** detaillent respectivement la realisation du backend et le module de securite.
- Le **chapitre 6** presente la realisation du frontend Angular.
- Le **chapitre 7** expose la strategie de tests et le deploiement Docker.
- Le **chapitre 8** relate les difficultes rencontrees et dresse le bilan quantitatif du travail.
- Une **conclusion generale** trace les perspectives d'evolution.

CHAPITRE 1

Contexte et cahier des charges

1.1. Contexte academique

Ce projet est realise dans le cadre du module Full-Stack du **Semestre 6 de la Licence Sciences Mathematiques et Informatique - option Developpement Informatique (DEV-INFO)**, dispense au Departement de Mathematiques et Informatique de la Faculte des Sciences Ben M'Sik. Ce module intervient en fin de cursus de licence, apres que les etudiants ont acquis les bases en programmation orientee objet, bases de donnees relationnelles, developpement web cote client et genie logiciel.

L'objectif pedagogique est triple :

- **Integrer** les acquis des modules precedents au sein d'un projet de taille significative.
- **Decouvrir** les frameworks de production utilises dans l'industrie (Spring Boot, Angular).
- **Pratiquer** le travail en binome sur un depot Git partage, en respectant des conventions de nommage et de commit.

1.2. Etude du marche de l'e-commerce au Maroc

Le commerce electronique marocain connait une expansion soutenue. Les plateformes leaders du marche - Jumia, Avito, Marjane.ma, Decathlon.ma - partagent un ensemble de fonctionnalites-types qui constituent l'*etat de l'art* : authentication utilisateur, navigation dans un catalogue categorise, recherche textuelle et par filtres, gestion d'un panier persistant, processus de commande securise, suivi d'historique, depot et consultation d'avis clients, gestion de favoris.

1.2.1. Analyse comparative

Le tableau ci-dessous resume une analyse comparative succincte des principales plateformes existantes au Maroc.

Critere	Forces communes	Faiblesses recurrentes
Catalogue	Richesse, pagination	Filtres parfois lents, peu pertinents
Panier	Persistence, modification quantite	Synchronisation multi-appareils inegale
Securite	HTTPS, JWT/OAuth	Mots de passe faibles autorises
Performance	CDN, cache	Surcharge en periode de soldes
Avis	Notation par etoiles	Moderation absente ou tardive

Tableau 1 : Analyse comparative succincte des plateformes existantes

E-Store ne pretend evidemment pas rivaliser avec ces geants, mais propose une maquette pedagogique fonctionnelle qui en reprend les fondamentaux dans un perimetre maitrisable.

1.3. Cahier des charges fonctionnel

1.3.1. Acteurs du systeme

L'application distingue deux profils d'utilisateurs :

- **USER** (utilisateur standard) : peut s'inscrire, se connecter, consulter le catalogue, ajouter au panier, passer commande, consulter son historique et déposer des avis.
- **ADMIN** (administrateur) : dispose, en plus, des droits de creation, modification et suppression sur les categories, les produits et le stock.

1.3.2. Cas d'usage principaux

Le tableau suivant recapitule les cas d'usage couverts :

Cas d'usage	Acteur	Description
Inscription	Visiteur	Email unique + mot de passe (≥ 6 caracteres)
Connexion	Utilisateur	Authentification, retour d'un JWT 24h
Profil	USER	Consultation et mise a jour du profil
Catalogue	Tous	Liste paginee, recherche, filtre categorie
Fiche produit	Tous	Detail, prix, stock, avis
Panier	USER	Ajout, modification, suppression, vidage
Commande	USER	Validation transactionnelle (ACID)
Avis	USER	Depot (1-5 etoiles + commentaire)
Categorie/Produit	ADMIN	CRUD complet
Stock	ADMIN	Lecture publique, mise a jour reservee

Tableau 2 : Cas d'usage fonctionnels couverts par E-Store

1.4. Cahier des charges non fonctionnel

1.4.1. Performance

Les pages doivent se charger en moins de **2 secondes** sur connexion nominale (4G ou Wi-Fi). Le bundle initial JavaScript doit rester sous **50 ko** grace au lazy loading.

1.4.2. Securite

- Mots de passe haches en **BCrypt** (jamais en clair).
- Authentification stateless par **JWT signe HMAC-SHA256**, expiration 24 heures.
- Endpoints d'administration proteges par **@PreAuthorize**.

- Validation systematique des entrees (Bean Validation).
- CORS restreint a l'origine localhost:4200 en developpement.

1.4.3. Maintainabilite

- Code conforme aux conventions Java/TypeScript.
- Decoupage en cinq domaines fonctionnels autonomes.
- Au moins **neuf tests unitaires** couvrant les services critiques.
- Documentation interne (Javadoc, README, commentaires).

1.4.4. Portabilite

- Demarrage possible **sans installation prealable** grace au profil dev (H2 in-memory).
- Demarrage Docker en une commande pour le profil prod.
- Compatibilite multi-OS (Windows, macOS, Linux).

1.5. Methodologie de travail

Nous avons retenu une approche **iterative legere**, inspiree d'Agile mais adaptee au contexte d'un binome etudiant. Le projet a ete decoupe en quinze etapes incrementales, chacune se concluant par un commit Git significatif. La repartition des roles a ete :

- **Akram Belmoussa** : Backend (Spring Boot, JPA, securite JWT), integration MongoDB, tests unitaires.
- **Nouhaila Ben Soumane** : Frontend (Angular 17, Bootstrap), modelisation des donnees, documentation et redaction du rapport.
- **Travail commun** : architecture, design de l'API, debogage, revue de code, integration finale.

CHAPITRE 2

Architecture et conception

2.1. Architecture generale en trois couches

E-Store adopte une architecture n-tiers classique en trois couches, qui presente l'avantage de separer clairement les responsabilites et de faciliter la maintenance. La figure 1 en donne une vue synthetique.

COUCHE PRESENTATION	Angular 17 standalone - Bootstrap 5 - TypeScript - RxJS / Signals
COUCHE LOGIQUE METIER	Spring Boot 3.3 - Controllers REST - Services - DTOs - Spring Security 6
COUCHE ACCES AUX DONNEES	Spring Data JPA - Spring Data MongoDB - MySQL 8 / H2 - MongoDB 7

Figure 1 : Architecture en trois couches de l'application E-Store

2.1.1. Couche presentation (frontend)

Cette couche est en charge de l'interaction avec l'utilisateur final. Elle n'a aucune connaissance du modele de persistance et communique avec le backend exclusivement via une **API REST/JSON**.

2.1.2. Couche logique metier (backend)

C'est le coeur applicatif. Elle recoit les requetes HTTP, valide les entrees, applique les regles de gestion (verification de stock, calcul de totaux, generation de jetons), orchestre les transactions et renvoie des DTOs serialises en JSON.

2.1.3. Couche d'accès aux données

Cette couche, masquee derriere les abstractions Spring Data, gere la persistance effective. Deux SGBD coexistent : **MySQL 8** (ou H2 en mode dev) pour les donnees strictement relationnelles (utilisateurs, produits, panier, commandes), et **MongoDB 7** pour les avis, donnees semi-structurees en forte croissance.

2.2. Decoupage en domaines fonctionnels

Chaque couche est elle-meme decoupee en **cinq domaines fonctionnels** cohesifs, suivant les principes du Domain-Driven Design. Un sixieme domaine, review, gere la particularite MongoDB.

customer User, Profile, JWT, Auth	catalog Category, Product, recherche	inventory Stock par produit
shopping Cart, CartItem	billing Order, OrderItem @Transactional	review Avis (MongoDB)

Figure 2 : Les six domaines fonctionnels de l'application

Cette double organisation *couches x domaines* produit la matrice suivante qui donne une vue exhaustive du projet :

Domaine	Presentation	Logique metier	Donnees
customer	login.component.ts	AuthService, JwtFilter	users, profiles (JPA)
catalog	product-list	ProductService, recherche	products, categories
inventory	Indicateur UI	InventoryService	inventories
shopping	cart.component	CartService	carts, cart_items
billing	orders.component	OrderService @Transactional	orders, order_items
review	review-form	ReviewService	MongoDB reviews

Tableau 2 : Matrice architecturale : 3 couches x 6 domaines

2.3. Modele conceptuel des donnees relationnel

Le modele relationnel comporte **neuf entites** reliees par des associations *one-to-one*, *one-to-many* et *many-to-one*. Les principales entites sont :

- **User** : id, firstName, lastName, email (unique), password (BCrypt), role
- **Profile** : id, phone, address, city, country (one-to-one User)
- **Category** : id, name, description (one-to-many Product)
- **Product** : id, name, description, price, imageUrl (many-to-one Category)
- **Inventory** : id, quantity (one-to-one Product)
- **Cart** : id, createdAt, updatedAt (one-to-one User)
- **CartItem** : id, quantity, unitPrice (many-to-one Cart, Product)
- **Order** : id, orderDate, totalAmount, status (many-to-one User)
- **OrderItem** : id, quantity, unitPrice (many-to-one Order, Product)

Les contraintes sont strictement respectees : email unique, prix en BigDecimal (jamais en double), cle etrangere NOT NULL sur les associations many-to-one obligatoires, cascade ALL avec orphanRemoval sur les collections de l'agregat.

2.4. Modele documentaire MongoDB

La collection reviews stocke les avis utilisateurs sous forme de documents JSON :

```
{
  "_id":      ObjectId("..."),
  "productId": 12,
  "userId":   5,
  "authorName": "Akram Belmoussa",
  "rating":   5,
  "comment":  "Excellent produit !",
  "createdAt": ISODate("2026-04-22T14:30:00Z")
}
```

Figure 4 : Structure d'un document de la collection reviews

Deux index secondaires sont crees via les annotations @Indexed sur les champs productId et userId, afin d'accelerer les requetes findByProductId et findByUserId.

2.5. Architecture de l'API REST

Toute communication frontend/backend transite par une API REST respectant les standards : methodes HTTP appropriees (GET / POST / PUT / DELETE), codes de retour normalises (200, 201, 204, 400, 401, 403, 404, 409), corps JSON encadre par une enveloppe ApiResponse standardisee, et versioning implicite par le prefixe /api/.

Verbe	URL	Description	Auth.
POST	/api/auth/register	Inscription	Public
POST	/api/auth/login	Connexion (retourne JWT)	Public
GET	/api/products	Catalogue + filtres	Public
GET	/api/products/{id}	Fiche produit	Public
POST	/api/products	Creation produit	ADMIN
GET	/api/categories	Liste categories	Public
GET	/api/cart	Panier courant	USER
POST	/api/cart/add	Ajout au panier	USER
POST	/api/orders	Validation commande	USER
GET	/api/orders	Historique	USER
POST	/api/reviews	Depot d'avis	USER
GET	/api/reviews/product/{id}	Avis d'un produit	Public

Tableau 3 : Endpoints REST principaux

CHAPITRE 3

Choix techniques et justifications

3.1. Backend : Java et Spring Boot

3.1.1. Pourquoi Java ?

Java demeure, en 2026, l'un des langages les plus largement deployes en entreprise - notamment dans le secteur bancaire, les telecommunications et le e-commerce de grande echelle. Ses atouts pedagogiques sont multiples : typage fort, programmation orientee objet stricte, ecosysteme mature, machine virtuelle performante. La version **Java 17 LTS** (Long-Term Support) a ete retenue pour sa stabilite et son adoption massive.

3.1.2. Pourquoi Spring Boot ?

Spring Boot est le framework Java de reference pour la construction d'applications backend modernes. Il automatise la configuration et fournit une myriade de *starters* prêts a l'emploi. Ses points forts sont :

- **Auto-configuration** par convention plutot que par configuration explicite.
- **Embedded Tomcat** : deploiement en JAR auto-executable.
- **Spring Data** : abstraction puissante des SGBD.
- **Spring Security 6** : securisation standardisee.
- **Actuator** : observabilite prete a l'emploi.

La **version 3.3** a ete choisie car compatible Jakarta EE 10 et Java 17+, et stable depuis fin 2024.

3.2. Persistence hybride : MySQL + MongoDB

Le choix d'un **double systeme de persistance** n'est pas un caprice : il repond a une realite du terrain. Toutes les donnees ne sont pas egales :

- Les **transactions financieres** (commandes, panier, stock) exigent les proprietes ACID : atomicite, coherence, isolation, durabilite. Le SQL relationnel y excelle.
- Les **avis utilisateurs** ne necessitent ni jointures complexes, ni transactions ; ils croissent rapidement et sont consultes massivement en lecture. Le NoSQL documentaire les sert plus efficacement.

Critere	MySQL 8 (relationnel)	MongoDB 7 (documentaire)
Paradigme	SQL, tables normalisees	BSON, documents JSON
Schema	Strict, predefini	Flexible

Transactions	ACID natives	Limitees (multi-doc v4+)
Joins	Performants	Rares (\$lookup)
Lectures massives	Bonnes (index)	Excellentes (replicas)
Cas d'usage E-Store	Users, Products, Orders	Reviews

Tableau 4 : Comparaison MySQL vs MongoDB dans le contexte E-Store

3.2.1. Bascule H2 / MySQL via les profils

Le projet exploite la fonctionnalite Spring de **profils** (application-dev.properties, application-prod.properties) pour basculer entre H2 in-memory (developpement) et MySQL persistant (production).

```
# Profil dev : H2 in-memory
spring.datasource.url=jdbc:h2:mem:estore;MODE=MySQL
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Profil prod : MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/estore
spring.datasource.username=root
spring.datasource.password=root
```

3.3. Frontend : Angular 17 standalone

3.3.1. Pourquoi Angular ?

Trois frameworks dominant le developpement web SPA en 2026 : **Angular**, **React** et **Vue**. Angular se distingue par son approche *tout-en-un* (router, formulaires, HTTP, i18n inclus) et son langage TypeScript natif, qui apporte la rigueur du typage statique au JavaScript. Pour un projet pedagogique de fin de module, Angular offre l'environnement le plus structurant.

3.3.2. Le mode standalone et les Signals

Depuis la version 14, et de maniere generalisee depuis la version 17, Angular permet de s'affranchir des NgModule historiques au profit de **composants standalone**. Les **signals**, nouveaute majeure d'Angular 17, remplacent avantageusement les Observable RxJS pour la gestion d'etat simple. Dans E-Store, ils servent par exemple a afficher dynamiquement le badge du panier dans la barre de navigation.

```
@Injectable({ providedIn: 'root' })
export class CartService {
  readonly cart = signal<Cart | null>(null);
  readonly itemCount = signal<number>(0);

  add(productId: number, qty: number) {
    return this.http.post(...).pipe(tap(r => this.update(r.data)));
  }
}
```

CHAPITRE 4

Realisation du backend

4.1. Structure generale du projet

L'arborescence `estore-backend/src/main/java/com/estore/` reflète fidelement le decoupage en domaines :

```
com.estore/
  EstoreApplication.java      <- Point d'entree
  customer/                   {entity, dto, repository, service, controller, security}
  catalog/                    {entity, dto, repository, service, controller}
  inventory/                  {entity, dto, repository, service, controller}
  shopping/                   {entity, dto, repository, service, controller}
  billing/                    {entity, dto, repository, service, controller}
  review/                     {document, dto, repository, service, controller}
  shared/                     ApiResponse.java
  config/                     CorsConfig, SecurityConfig, DataSeeder
  exception/                  GlobalExceptionHandler + 3 exceptions metier
```

Chaque domaine est **autonome** : il ne peut dependre que des couches inferieures de son propre domaine, ou des composants partages. Les domaines communiquent exclusivement via leurs services publics.

4.2. Domaine customer (User, Profile, JWT)

Ce domaine pivote autour de l'entite User, a laquelle est associe un Profile en relation @OneToOne.

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @JsonIgnore
    @Column(nullable = false)
    private String password;           // hash BCrypt, jamais en clair

    @Enumerated(EnumType.STRING)
    private Role role;                 // USER ou ADMIN

    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.LAZY)
    private Profile profile;
}
```

L'annotation `@JsonIgnore` sur `password` empeche toute serialisation accidentelle. La cascade ALL sur `profile` garantit l'atomicite creation/suppression.

4.2.1. AuthService

L'AuthService centralise la logique d'inscription et de connexion. La connexion delegue a Spring Security via l'AuthenticationManager, puis genere un JWT.

```
@Transactional
public AuthResponseDto register(RegisterDto dto) {
    if (userRepository.existsByEmail(dto.getEmail()))
        throw new BusinessException("Cet email est deja utilise");

    User user = User.builder()
        .firstName(dto.getFirstName()).lastName(dto.getLastName())
        .email(dto.getEmail())
        .password(passwordEncoder.encode(dto.getPassword())) // BCrypt
        .role(Role.USER).build();

    user.setProfile(Profile.builder().user(user).build());
    User saved = userRepository.save(user);

    String token = jwtService.generateToken(saved);
    return AuthResponseDto.builder()
        .token(token).user(UserDto.from(saved)).build();
}
```

4.3. Domaine catalog

Le catalogue regroupe Category et Product. Le repository ProductRepository expose une methode personnalisee combinant recherche textuelle et filtre par categorie :

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    @Query("""
        SELECT p FROM Product p
        WHERE (:categoryId IS NULL OR p.category.id = :categoryId)
        AND (:q IS NULL OR LOWER(p.name) LIKE LOWER(CONCAT('%', :q, '%'))
            OR LOWER(p.description) LIKE LOWER(CONCAT('%', :q, '%')))
        """)
    Page<Product> search(@Param("categoryId") Long categoryId,
        @Param("q") String q, Pageable pageable);
}
```

L'utilisation de JPQL plutot que de SQL natif garantit la portabilite H2/MySQL, et la pagination via Pageable evite le retour massif d'enregistrements.

4.4. Domaine inventory

L'entite Inventory est mappee 1-1 avec Product. Le service expose deux operations essentielles : `checkAvailability` (lecture sans modification) et `decrement` (verification + decrement). Cette separation garantit que la verification amont du checkout ne modifie pas le stock.

4.5. Domaines shopping et billing

4.5.1. Le panier (shopping)

L'entite Cart maintient une relation @OneToMany avec CartItem. A l'ajout d'un produit deja present, la quantite est incrementee plutot que dupquee.

4.5.2. Le checkout transactionnel (billing)

La validation de commande constitue **l'operation la plus critique** de l'application. Elle touche trois agregats (Cart, Order, Inventory) et doit etre strictement atomique.

```
@Transactional
public OrderDto checkout() {
    User user = securityUtils.currentUser();
    Cart cart = cartService.getOrCreateCart(user);
    if (cart.getItems().isEmpty())
        throw new BusinessException("Votre panier est vide");

    // 1) Verifier le stock pour TOUS les items AVANT toute modification
    for (CartItem ci : cart.getItems())
        inventoryService.checkAvailability(
            ci.getProduct().getId(), ci.getQuantity());

    // 2) Creer la commande + decremter le stock
    Order order = Order.builder().user(user).status(PENDING).build();
    BigDecimal total = BigDecimal.ZERO;
    for (CartItem ci : cart.getItems()) {
        order.getItems().add(OrderItem.builder()...build());
        total = total.add(ci.getUnitPrice()
            .multiply(BigDecimal.valueOf(ci.getQuantity())));
        inventoryService.decrement(
            ci.getProduct().getId(), ci.getQuantity());
    }
    order.setStatus(CONFIRMED);

    // 3) Sauvegarder + vider le panier
    Order saved = orderRepository.save(order);
    cartService.clearCart(cart);
    return OrderDto.from(saved);
}
```

L'annotation @Transactional garantit qu'en cas d'exception levee a n'importe quelle etape, toutes les modifications faites depuis l'entree dans la methode sont automatiquement annulees (*rollback*). C'est l'atomicite ACID en action.

4.6. Domaine review (MongoDB)

Le document Review est mappe sur la collection MongoDB eponyme :

```
@Document(collection = "reviews")
public class Review {
    @Id private String id;
    @Indexed private Long productId;
    @Indexed private Long userId;
    private String authorName;
    private int rating;
    private String comment;
    private Instant createdAt;
}

public interface ReviewRepository extends MongoRepository<Review, String> {
    List<Review> findByProductIdOrderByCreatedAtDesc(Long productId);
}
```

```
List<Review> findByUserIdOrderByCreatedAtDesc(Long userId);
}
```

Le repository tire profit de la *convention over configuration* de Spring Data : la simple declaration d'une methode au nom evocateur produit la requete correspondante.

4.7. Gestion globale des erreurs

Plutot que de laisser remonter des exceptions Java brutes vers le client, l'application centralise leur traitement via un `@RestControllerAdvice`. Chaque exception metier est convertie en reponse JSON propre avec le code HTTP approprie.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>> handleNotFound(...) {
        return ResponseEntity.status(NOT_FOUND)
            .body(ApiResponse.error(ex.getMessage()));
    }

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ApiResponse<Void>> handleBusiness(...) {
        return ResponseEntity.status(CONFLICT)
            .body(ApiResponse.error(ex.getMessage()));
    }
    // ... + ValidationException, BadCredentials, AccessDenied
}
```

CHAPITRE 5

Securite de l'application

La securite de toute application e-commerce est une exigence absolue. E-Store met en oeuvre une chaine complete depuis le hachage des mots de passe jusqu'a l'autorisation fine par role.

5.1. Hachage BCrypt des mots de passe

Aucun mot de passe utilisateur n'est jamais stocke en clair. Tous transitent par l'algorithme **BCrypt**, un *one-way hash* avec sel aleatoire et facteur de cout ajustable. Le facteur 10 (par default Spring) est utilise, soit $2^{10} = 1024$ rounds.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

5.2. Authentification stateless par JWT

L'authentification suit un modele *stateless* : aucun etat n'est conserve cote serveur entre deux requetes. Le client porte lui-meme la preuve de son identite, sous la forme d'un JSON Web Token signe.

5.2.1. Structure d'un JWT

Un JWT est compose de trois parties separees par un point : header.payload.signature. Le payload de E-Store contient les *claims* suivants :

- sub : email de l'utilisateur (subject)
- uid : identifiant numerique
- role : USER ou ADMIN
- name : nom complet (pour affichage UI)
- iat et exp : timestamps de creation et d'expiration

5.2.2. Cycle de vie complet

#	Etape	Composant
1	Client envoie email + password	POST /api/auth/login
2	Validation BCrypt	passwordEncoder.matches()
3	Generation du token (HS256, 24h)	JwtService
4	Stockage cote client	localStorage 'estore.token'

5	Toute requete : Bearer <token>	AuthInterceptor
6	Validation a chaque appel	JwtAuthenticationFilter
7	Population SecurityContext	SecurityContextHolder

Figure 5 : Cycle de vie d'un JWT dans E-Store

5.2.3. Generation et validation du token

```
public String generateToken(User user) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("uid", user.getId());
    claims.put("role", user.getRole().name());
    claims.put("name", user.getFirstName() + " " + user.getLastName());

    Date now = new Date();
    Date exp = new Date(now.getTime() + expirationMs); // 24h

    return Jwts.builder()
        .claims(claims).subject(user.getEmail())
        .issuedAt(now).expiration(exp)
        .signWith(key) // HMAC-SHA256
        .compact();
}
```

5.3. Autorisation par role

Les operations sensibles (creation/modification/suppression de categories et de produits, mise a jour du stock) sont protegees par `@PreAuthorize` :

```
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<ApiResponse<ProductDto>> create(
    @Valid @RequestBody CreateProductDto dto) {
    return ResponseEntity.status(CREATED)
        .body(ApiResponse.ok("Produit cree", productService.create(dto)));
}
```

Tout appel a cet endpoint par un utilisateur non-ADMIN se solde par un HTTP 403 Forbidden, intercepte par le `GlobalExceptionHandler`.

5.4. Configuration CORS

En developpement, le frontend tourne sur le port 4200 et le backend sur le port 8080. Cette difference d'origine declencherait une erreur CORS du navigateur sans configuration appropriee.

```
@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/api/**")
        .allowedOrigins("http://localhost:4200")
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("*").allowCredentials(true);
}
```

CHAPITRE 6

Realisation du frontend

6.1. Structure de l'application Angular

Le projet Angular suit l'organisation **core / shared / features**, classique des applications Angular modernes :

```
src/app/
  app.component.ts      <- Composant racine
  app.config.ts         <- Configuration globale (providers)
  app.routes.ts         <- Routes
  core/
    services/           <- AuthService, CartService, ...
    guards/             <- AuthGuard
    interceptors/       <- AuthInterceptor, ErrorInterceptor
    models/             <- Interfaces TypeScript
  shared/
    components/         <- Header, Footer, Loader, Toast
  features/
    auth/               <- Login, Register
    catalog/            <- ProductList, ProductDetail
    cart/               <- Cart
    orders/             <- Orders
    profile/            <- Profile
    reviews/           <- ReviewForm
```

6.2. Bootstrap de l'application

Le fichier `main.ts` amorce l'application sans `NgModule`, en passant directement par `bootstrapApplication` :

```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig)
  .catch(err => console.error(err));

// app.config.ts - configuration centralisee
export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([
      authInterceptor, errorInterceptor
    ]))
  ]
};
```

6.3. Routing et lazy loading

Toutes les routes utilisent `loadComponent` pour un lazy loading natif, ce qui produit des bundles separees par fonctionnalite. Mesure : le bundle initial pese **13.84 ko**, et chaque feature ajoute en moyenne **8 ko** charges a la demande.

```
export const routes: Routes = [
  { path: '', loadComponent: () =>
    import('./features/catalog/product-list.component')
    .then(m => m.ProductListComponent) },
  { path: 'login', loadComponent: () =>
    import('./features/auth/login.component')
    .then(m => m.LoginComponent) },
  { path: 'cart', loadComponent: () =>
    import('./features/cart/cart.component')
    .then(m => m.CartComponent),
    canActivate: [authGuard] }
];
```

6.4. AuthInterceptor et ErrorInterceptor

L'**AuthInterceptor** attache silencieusement le JWT a chaque requete HTTP sortante. L'**ErrorInterceptor** centralise la gestion d'erreurs : 401 entraine la deconnexion automatique, 4xx/5xx affichent un toast.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token;
  if (!token) return next(req);
  return next(req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  }));
};

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const toast = inject(ToastService);
  const auth = inject(AuthService);
  const router = inject(Router);

  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) {
        auth.logout();
        router.navigate(['/login']);
        toast.error('Session expiree');
      } else if (err.status !== 0) {
        toast.error(err.error?.message || `Erreur ${err.status}`);
      }
      return throwError(() => err);
    })
  );
};
```

6.5. AuthGuard et composants

Trois routes sont protegees par `authGuard` : `/cart`, `/orders` et `/profile`. Le panier d'un visiteur non connecte n'est donc jamais persiste cote serveur.

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
```

```
    if (auth.isAuthenticated) return true;
    router.navigate(['/login']);
    return false;
};
```

L'interface adopte les codes du e-commerce moderne : barre de navigation fixe en tete, grille de cartes produits responsive (1 / 2 / 4 colonnes selon la largeur d'ecran), badges colores indiquant la disponibilite stock. Bootstrap 5 fournit l'ossature CSS et les composants standards (modales, alertes, formulaires).

CHAPITRE 7

Tests et deployment

7.1. Strategie de tests

Suivant la **pyramide de tests** de Mike Cohn, nous avons concentre l'effort sur la base de la pyramide : les tests unitaires des services metier. Ces tests sont rapides, deterministes et faciles a maintenir.

Outils utilises :

- **JUnit 5** (Jupiter) : framework de test
- **Mockito 5** : isolation des dependances
- **AssertJ** : assertions fluentes (assertThat...)
- **Spring Boot Test** : tests d'integration eventuels

Suite	Tests	Statut	Couverture
ProductServiceTest	4	PASS	findById (present/absent), recherche (avec/sans mot-cle)
CartServiceTest	2	PASS	Ajout avec stock suffisant et insuffisant
OrderServiceTest	3	PASS	Panier vide, validation complete, historique
TOTAL	9	0 failure	BUILD SUCCESS

Tableau 5 : Suite de tests unitaires E-Store

7.2. Resultats d'execution

```
$ ./mvnw test
[INFO] Tests run: 3, Failures: 0, Errors: 0 -- OrderServiceTest
[INFO] Tests run: 4, Failures: 0, Errors: 0 -- ProductServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0 -- CartServiceTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Figure 6 : Resultats d'execution des tests JUnit

7.3. Conteneurisation Docker

Le fichier docker-compose.yml orchestre quatre services :

```
services:
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: estore
```

```
    ports: ["3306:3306"]
  mongo:
    image: mongo:7
    ports: ["27017:27017"]
  phpmyadmin:
    image: phpmyadmin:latest
    ports: ["8081:80"]
  mongo-express:
    image: mongo-express:latest
    ports: ["8082:8081"]
```

Figure 7 : Organisation des services Docker Compose

Une commande suffit alors a provisionner l'environnement complet : `docker compose up -d`.

7.3.1. Demarrage en trois commandes

```
# 1. Bases de donnees (optionnel : profil prod uniquement)
docker compose up -d

# 2. Backend
cd estore-backend && ./mvnw spring-boot:run

# 3. Frontend
cd estore-frontend && npm install && npm start

# 4. Ouvrir http://localhost:4200
```

7.3.2. Versionnement Git et publication GitHub

L'historique du projet est versionne par Git, organise en **21 commits** semantiques (feat, fix, docs, chore). Le projet est publie sur GitHub a l'adresse github.com/akrambelmoussa-etu-byte/e-store.

CHAPITRE 8

Difficultes rencontrees et resultats

Le developpement n'a pas ete un long fleuve tranquille. Plusieurs obstacles techniques significatifs ont ete rencontres ; les surmonter a ete l'occasion d'approfondir notre comprehension de l'ecosysteme.

8.1. Difficultes rencontrees

8.1.1. Strictness Mockito

Lors du premier lancement de mvn test, deux tests echouent avec le message *UnnecessaryStubbingException: Unnecessary stubbings detected.*

Cause : Mockito 5 applique par default le mode strict. Toute instruction `when(...).thenReturn(...)` declaree mais non verifiee durant l'execution du test provoque une erreur.

Solution : annoter chaque classe de test avec `@MockitoSettings(strictness = Strictness.LENIENT)`, ce qui relache la verification tout en conservant la securite d'ecriture.

8.1.2. LazyInitializationException

Lors du retour d'une entite User via le controleur, Hibernate levait une *LazyInitializationException* sur le Profile charge en mode LAZY.

Cause : la serialisation Jackson tentait de naviguer dans la relation apres la fermeture de la session JPA.

Solution : adoption systematique du pattern **DTO**. Chaque endpoint renvoie un objet de transfert (UserDetailsDto), construit dans la couche service alors que la session est encore ouverte. Avantage collaterale : les DTOs masquent les champs sensibles (password) et stabilisent l'API.

8.1.3. Installation de Docker Desktop

Lors de l'installation de Docker Desktop sous Windows 11, l'erreur suivante bloque le processus :

« For security reasons C:\ProgramData\DockerDesktop must be owned by an elevated account. »

Cause : un dossier residuel d'une installation anterieure subsistait avec des permissions incorrectes.

Solutions implementees :

- **Solution A (brute) :** suppression manuelle en mode administrateur via `rmdir /s /q`, puis reinstallation propre.

- **Solution B (elegante)** : mise en place d'un **profil dev** base sur H2 in-memory, qui permet de demarrer l'application sans aucune installation prealable. Cette dualite Docker/H2 a ete pensee des la conception et a fait ses preuves le jour de la demonstration.

8.1.4. Encodage des tokens JWT

Lors de la generation du premier JWT, la bibliotheque jjwt levait une WeakKeyException : la cle secrete configuree comportait moins de 256 bits.

Solution : implementer une logique de fallback : tenter d'abord un decodage Base64 de la cle, et en cas d'echec ou de longueur insuffisante, utiliser l'encodage UTF-8 brut.

```
byte[] bytes;
try {
    bytes = Decoders.BASE64.decode(secret);
    if (bytes.length < 32) throw new IllegalArgumentException();
} catch (Exception e) {
    bytes = secret.getBytes(StandardCharsets.UTF_8);
}
this.key = Keys.hmacShaKeyFor(bytes);
```

8.2. Bilan fonctionnel et metriques

L'application livree couvre integralement le perimetre du cahier des charges. Le tableau suivant en synthetise les fonctionnalites :

Fonctionnalite	Description	Statut
Inscription / Connexion	Email + mot de passe, JWT 24h	Livre
Profil utilisateur	Consultation et modification	Livre
Catalogue pagine	Recherche, filtre par categorie	Livre
Fiche produit	Detail, image, stock, avis	Livre
Panier persistant	Ajout, modification, vidage	Livre
Validation commande	Transactionnelle ACID	Livre
Historique commandes	Consultation detaillee	Livre
Avis produits	Depot et consultation (MongoDB)	Livre
Administration	CRUD categories / produits / stock	Livre

Tableau 7 : Bilan fonctionnel des livrables

Quelques metriques quantitatives :

Indicateur	Valeur
Lignes de code Java	~ 3 500
Lignes de code TypeScript	~ 2 000
Nombre de classes Java	71

Nombre de composants Angular	14
Nombre d'endpoints REST	24
Tests unitaires	9 (100% passants)
Bundle JS initial	13.84 ko
Temps de demarrage backend (H2)	≈ 4 secondes
Temps de reponse API (mediane)	≈ 30 ms
Commits Git	21

Tableau 6 : Metriques quantitatives du projet

CHAPITRE CONCL

Conclusion generale et perspectives

1. Bilan

Au terme de ce projet, nous avons mene a bien la conception et la realisation d'une application e-commerce **integralement fonctionnelle**, couvrant le parcours utilisateur du visiteur anonyme jusqu'a la commande validee. L'architecture en **trois couches x cinq domaines** a ete rigoureusement respectee, fournissant une base saine pour toute evolution ulterieure.

Sur le plan technique, les objectifs initiaux ont tous ete atteints :

- Architecture full-stack moderne avec Spring Boot 3.3 et Angular 17.
- Persistance hybride MySQL + MongoDB operationnelle.
- Securite robuste par BCrypt + JWT + @PreAuthorize.
- Suite de tests JUnit 5 / Mockito en BUILD SUCCESS.
- Conteneurisation Docker Compose, profil de fallback H2.

Sur le plan humain, le travail en binome nous a permis d'experimenter *in vivo* les pratiques du developpement collaboratif : versionnement Git, conventions de commit, revue de code, repartition des taches, communication asynchrone.

2. Apports personnels

Au-dela de la technique, ce projet nous a confronte a la **realite de l'incertitude** qui caracterise tout developpement logiciel : choix d'architecture aux consequences durables, debogage de problemes inattendus, arbitrages entre elegance du code et delais. Apprendre a composer avec cette incertitude, plutot qu'a la fuir, est probablement le benefice le plus durable de cette experience.

3. Perspectives

Le projet livre n'est pas un produit fini mais un **socle evolutif**. Plusieurs pistes d'extension ont ete identifiees :

- **Paiement en ligne** : integration de Stripe ou des passerelles marocaines (CMI, Maroc Telecommerce).
- **Notifications transactionnelles** : envoi automatique d'emails (Spring Mail) ou de SMS (Twilio) lors de la confirmation de commande.
- **Recommandations** : suggestion produits basee sur l'historique d'achat (collaborative filtering).

- **Internationalisation** : passage du français à l'arabe ou l'anglais via Angular i18n.
- **Deploiement cloud** : conteneurisation et déploiement sur Heroku, Render ou AWS Free Tier.
- **Application mobile** : portage Ionic ou Flutter consommant la même API.
- **Observabilité** : intégration Prometheus + Grafana pour les métriques applicatives.

Plus fondamentalement, l'architecture en couches et en domaines que nous avons mise en place rend ces évolutions **additives** : il sera possible de les introduire sans casser l'existant. C'est, à nos yeux, la preuve la plus convaincante que les choix architecturaux de départ étaient les bons.

CHAPITRE BIB

Bibliographie et webographie

Ouvrages :

- [1] Craig Walls, *Spring Boot in Action*, 2nd edition, Manning Publications, 2024.
- [2] Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.
- [3] Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- [4] Martin Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.

Documentations officielles :

- [5] Spring Framework Reference - docs.spring.io/spring-framework/
- [6] Spring Boot Reference - docs.spring.io/spring-boot/
- [7] Spring Data JPA - docs.spring.io/spring-data/jpa/
- [8] Spring Data MongoDB - docs.spring.io/spring-data/mongodb/
- [9] Angular Documentation - angular.dev
- [10] MongoDB Manual - mongodb.com/docs/manual/
- [11] RFC 7519 - JSON Web Token (JWT) - rfc-editor.org/rfc/rfc7519
- [12] OWASP Authentication Cheat Sheet - cheatsheetseries.owasp.org

Articles et tutoriels :

- [13] Baeldung - A Guide to JPA with Spring
- [14] Baeldung - Spring Security with JWT
- [15] Angular Blog - Introducing Angular Signals

CHAPITRE ANN

Annexes

A. Configuration application.properties

```
# === application.properties (commun) ===
spring.application.name=estore-backend
spring.profiles.active=dev
server.port=8080

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false

# MongoDB
spring.data.mongodb.uri=mongodb://localhost:27017/estore

# JWT
jwt.secret=change-me-with-a-256-bits-secret-key-for-production
jwt.expiration=86400000
```

B. Comptes de test

Le DataSeeder cree automatiquement deux comptes au premier demarrage si la base est vide :

Role	Email	Mot de passe
ADMIN	admin@estore.ma	Admin@123
USER	user@estore.ma	User@123

C. Procedure de demarrage rapide

```
# 1. Cloner le depot
git clone https://github.com/akrambelmoussa-etu-byte/e-store.git
cd e-store

# 2a. Demarrer en mode dev (H2 + Mongo optionnel)
cd estore-backend
./mvnw spring-boot:run

# 2b. (Alternative) Demarrer en mode prod
docker compose up -d
cd estore-backend
./mvnw spring-boot:run -Dspring.profiles.active=prod

# 3. Frontend (terminal separe)
cd estore-frontend
npm install
npm start

# 4. Ouvrir http://localhost:4200
```